



AI-Driven Lab Exam Monitoring and Management System

Zain Ali Mehmood (2280133)
Muhammad Haider Ali (2280354)
Roshaan Ali Shah (2280327)

Supervised by: Ms. Zainab iftikhar

In partial fulfillment of requirement for the degree
Bachelor of Science (Software Engineering)
Shaheed Zulfikar Ali Bhutto Institute of Science and Technology
Islamabad, Pakistan

Spring, 2026

AI-Driven Lab Exam Monitoring and Management System

By

Zain Ali Mehmood (2280133)
Muhammad Haider Ali (2280354)
Roshan Ali Shah (2280327)

CERTIFICATE

It is to certify that the above students thesis has been completed to my satisfaction and, to my belief, its standard is appropriate for submission for Evaluation. I have also conducted plagiarism test of this thesis using HEC prescribed software and found similarity index "write plagiarism percentage" that is within the permissible limit set by the HEC for the BSSE degree thesis. I have also found the thesis in a format recognized by the SZABIST for the BSSE thesis.

(Supervisor)
Ms. Zainab Iftikhar

(FYP Committee Head)
Name

Date: _____

Department of Computer Science, Shaheed Zulfikar Ali Bhutto Institute of
Science and Technology, Islamabad Campus

Spring, 2026

DECLARATION

We, Zain Ali Mehmood (2280133), Muhammad Haider Ali (2280354), and Roshaan Ali Shah (2280327), students of Bachelor of Science in Software Engineering at Shaheed Zulfiqar Ali Bhutto Institute of Science and Technology, Islamabad, hereby declare that the project titled “AI-Driven Lab Exam Monitoring and Management System” is our own work.

This project has not been submitted previously for any degree or examination at any other university or institute. All sources of information and data used in this document have been duly acknowledged.

Zain Ali Mehmood
(2280133)

Muhammad Haider Ali
(2280354)

Roshaan Ali Shah
(2280327)

Project Overview

The proposed project, AI-Driven Lab Exam Monitoring and Management System, addresses the need for a controlled and well-managed environment for practical laboratory examinations conducted in universities and other academic institutions. In courses such as programming, databases, software engineering, networking, and operating systems, students are assessed on institutional machines where academic integrity depends on secure access, proper scheduling, controlled workstation usage, continuous monitoring, organized submission handling, and dependable reporting. Existing tools usually address only one part of this process, such as browser lockdown, online quiz delivery, plagiarism detection, or grading support, but they do not provide one integrated workflow specifically designed for in-person lab examinations.

The scope of the project is to develop a unified academic platform that supports exam scheduling, user and role management, lab allocation, student authentication, face recognition support, AI-assisted monitoring, evidence logging, submission collection, result management, and report generation. The system is intended to reduce manual invigilation burden, improve institutional visibility during live exams, and maintain a stronger evidence trail for post-exam review. By combining these functions in one platform, the project aims to improve fairness, consistency, and accountability in practical assessments.

The development approach of the project is iterative and modular because the solution contains administrator, teacher, and student workflows along with backend services, database support, monitoring functions, and reporting modules. The expected outcome is a practical prototype that can help institutions manage the complete lifecycle of a lab exam more effectively, from pre-exam planning to post-exam analysis, while also serving as a foundation for future academic process automation.

Dedication

Firstly, we dedicate this project to Allah Almighty, whose blessings, mercy, and guidance made this effort possible. We also dedicate this work to our Holy Prophet Muhammad (Peace Be Upon Him), whose teachings remain a source of inspiration in all aspects of life. Finally, we dedicate this project to our beloved parents, respected teachers, supervisor, and supportive friends whose prayers, encouragement, and sacrifices have helped us continue our academic journey with determination and hope.

Acknowledgement

This Final Year Project has been completed in partial fulfillment of the requirements for the degree of Bachelor of Science in Software Engineering. We sincerely thank our supervisor, Ms. Zainab Iftikhar, for her guidance, encouragement, and constructive feedback throughout the development of this project. We are also grateful to the faculty members of the Department of Computer Science, SZABIST Islamabad, and our families and friends for their continuous support, motivation, and prayers during the planning, implementation, and report-writing phases of this work.

Revision History

Compiled By	Checked By	Date	Description	Version
M.Haider Ali	Mam Zainab Iftikhar Chaudhary	10 Mar 2026	Section 1.0	1.0
Roshaan Ali shah	Mam Zainab Iftikhar Chaudhary	12 Mar 2026	Section 1.1	1.1
Zain Ali Mehmood	Mam Zainab Iftikhar Chaudhary	14 Mar 2026	Section 1.2	1.2
M.Haider Ali	Mam Zainab Iftikhar Chaudhary	16 Mar 2026	Section 1.3	1.3
Roshaan Ali shah	Mam Zainab Iftikhar Chaudhary	18 Mar 2026	Section 1.4, 1.5	1.4
Zain Ali Mehmood	Mam Zainab Iftikhar Chaudhary	20 Mar 2026	Section 1.6	1.5
M.Haider Ali	Mam Zainab Iftikhar Chaudhary	22 Mar 2026	Section 1.7	1.6
Roshaan Ali shah	Mam Zainab Iftikhar Chaudhary	24 Mar 2026	Section 1.8	1.7
Zain Ali Mehmood	Mam Zainab Iftikhar Chaudhary	26 Mar 2026	Section 2.0	1.8
M.Haider Ali	Mam Zainab Iftikhar Chaudhary	28 Mar 2026	Section 2.1	1.9
Roshaan Ali shah	Mam Zainab Iftikhar Chaudhary	30 Mar 2026	Section 2.2	2.0
Zain Ali Mehmood	Mam Zainab Iftikhar Chaudhary	01 Apr 2026	Section 3.0	2.1
M.Haider Ali	Mam Zainab Iftikhar Chaudhary	03 Apr 2026	Section 3.1	2.2
Roshaan Ali shah	Mam Zainab Iftikhar Chaudhary	05 Apr 2026	Section 3.2	2.3

Contents

Project Overview	ii
Dedication	iii
Acknowledgements	iv
Revision History	
List of Figures	iii
List of Tables	1
1 Introduction	2
1.1 Product Scope	2
1.1.1 Existing System Description	2
1.1.2 Future System Usage Analysis	7
1.2 Objectives	8
1.3 Problem Statement / Limitations	8
1.4 Benefits of Proposed System	9
1.5 Intended Market of Product	10
1.5.1 Target Audience	10
1.5.2 Institutional Environment	10
1.6 Intended Users of the Product	10
1.6.1 Administrator	10
1.6.2 Teachers and Students	10
1.7 Software Process Model	11
1.7.1 Process Model Introduction	11
1.7.2 Justification of Proposing the Process Model	12
1.7.3 Steps of Process Model	12
2 SOFTWARE REQUIREMENTS SPECIFICATION	14
2.1 Introduction	14
2.1.1 Document Scope	15
2.1.2 Audience	15
2.2 Functional Requirements	15
2.2.1 User and Access Management Requirements	16
2.2.2 Examination Planning and Scheduling Requirements	17
2.2.3 Monitoring and Supervision Requirements	17
2.2.4 Submission and Evidence Requirements	17
2.2.5 Reporting and Post-Exam Review Requirements	17
2.3 Non-Functional Requirements	18
2.3.1 Software Quality Attributes	18
2.3.2 Other Non-Functional Requirements	18
2.4 Requirement Gathering Techniques Used	19
2.5 Time Frame	21

3	SOFTWARE PROJECT PLAN	22
3.1	Deliverables of the Project	22
3.1.1	Requirements and Analysis Deliverables	22
3.1.2	Design Deliverables	23
3.1.3	Prototype and Module Deliverables	23
3.1.4	Testing and Validation Deliverables	23
3.1.5	Documentation and Presentation Deliverables	23
3.1.6	Gantt Chart	23
3.1.7	Work Breakdown Structure	24
3.1.8	Critical Path Method	25
3.2	Project Risk Management	26
3.2.1	Risk Identification	26
3.2.2	Risk Analysis	26
3.2.3	Risk Mitigation Plan	27
3.2.4	Risk Monitoring and Control	27

List of Figures

1.1	Detailed Agile Workflow with Sprint Feedback and Review Loops	11
3.1	Project Gantt Chart Based on Eight Agile Sprints	24
3.2	Work Breakdown Structure of the Proposed Project	25
3.3	Critical Path of the Proposed Project Activities	26

List of Tables

1.1	Abbreviations	3
1.2	Application Comparison	7
2.1	Functional Requirements	16
2.2	Non-Functional Requirements	19
2.3	Project Time Frame (8 Sprints)	21
3.1	Risk Likelihood and Impact Assessment	27

Chapter 1

Introduction

The growing dependence of higher education on digital infrastructure has transformed not only classroom delivery but also the way assessment is planned, conducted, and evaluated. In computer science and software engineering programs, practical laboratory examinations occupy a central place because they test problem solving, coding ability, system usage skills, and real-time decision-making in controlled academic settings. However, many institutions still conduct such examinations through fragmented workflows in which exam scheduling, laboratory readiness, invigilation, student attendance, suspicious-activity monitoring, submission collection, and post-exam reporting are handled using separate manual or semi-digital methods. This fragmentation reduces operational clarity and creates avoidable pressure on instructors and invigilators during already time-sensitive exam sessions. For this reason, the proposed AI-Driven Lab Exam Monitoring and Management System has been conceived as an integrated academic platform that improves visibility, control, traceability, and institutional reliability throughout the full life cycle of a laboratory examination.

The idea of combining examination management with monitoring support emerges from a practical institutional need. In many university labs, supervisors are required to observe multiple students simultaneously, enforce timing rules, answer operational issues, verify candidate identity, track attendance, and later justify the validity of the conducted exam. When these responsibilities are distributed across paper lists, messaging groups, manual observation, local file collection, and after-the-fact reporting, the academic process becomes difficult to standardize. Errors are more likely to occur, evidence is harder to retain, and the fairness of the assessment may become difficult to demonstrate. A unified digital system can therefore provide an important institutional benefit by connecting planning, monitoring, secure execution, submission handling, and reporting into one auditable workflow.

1.1 Product Scope

The scope of the proposed AI-Driven Lab Exam Monitoring and Management System is limited to the planning, controlled execution, monitoring, submission handling, and reporting of practical laboratory examinations in an academic environment. The proposed system covers twelve project modules: user authentication, role management, exam scheduling, question and instruction management, laboratory allocation, workstation control, student attendance, face recognition, AI-based monitoring, incident logging, result management, and reporting. Within this scope, the product delivers five core feature groups: secure role-based access for administrators, teachers, and students; scheduled exam creation with lab and machine mapping; monitored exam execution with face verification and AI-supported alerts; organized submission, result, and evidence handling; and administrative reporting for review, audit, and decision support.

1.1.1 Existing System Description

The current practical laboratory examination environment in many academic institutions is still managed through a mix of manual supervision, isolated software tools, verbal coordination, and post-exam documentation. Instructors often prepare the exam separately, communicate schedules through informal channels, verify attendance manu-

Table 1.1: Abbreviations

Serial #	Abbreviation	Definition
1	AI	Artificial Intelligence
2	LMS	Learning Management System
3	CBT	Computer-Based Testing
4	SRS	Software Requirements Specification
5	RBAC	Role-Based Access Control
6	UI	User Interface
7	UX	User Experience
8	WBS	Work Breakdown Structure
9	CPM	Critical Path Method
10	QA	Quality Assurance
11	DBMS	Database Management System
12	API	Application Programming Interface

ally, collect files from multiple sources, and later reconstruct incidents from memory or incomplete notes. This fragmented setup becomes difficult to manage when the number of students, workstations, and simultaneous lab sessions increases.

The following application review highlights ten widely used systems related to online assessment, secure testing, plagiarism analysis, and learning management. Each review shows that existing tools solve selected parts of the examination problem, but none of them fully combines lab scheduling, identity verification, machine control, AI-assisted monitoring, result management, and institutional reporting in one workflow tailored for supervised lab exams.

1.1.1.1 Respondus LockDown Browser Respondus LockDown Browser is a dedicated assessment browser that helps institutions protect online tests by restricting access to other applications, browser functions, printing options, and copy-paste behavior during an exam session [4]. It is widely used in universities because it integrates with popular learning platforms and focuses on maintaining a controlled test-taking interface.

Features

- Integrates with major learning management systems.
- Restricts opening of new tabs and applications.
- Disables printing and copy-paste actions.
- Supports controlled quiz launch processes.
- Offers add-on monitoring support through Respondus Monitor.

Limitations

- Does not provide complete lab scheduling workflows.
- Offers limited institution-specific workstation mapping.
- Does not focus on result and report management for lab exams.
- Incident handling is narrower than a dedicated invigilator dashboard.
- Practical file-based lab workflows require additional systems.

1.1.1.2 Safe Exam Browser Safe Exam Browser is an open-source secure browser environment that turns a computer into a locked-down workstation for e-assessments [5].

Features

- Creates a locked-down exam workstation.
- Supports configurable exam settings.
- Restricts navigation outside the exam environment.
- Works with supported assessment platforms.
- Is available for multiple operating systems.

Limitations

- Does not provide full exam administration workflows.
- Lacks dedicated result management functions.
- Does not include built-in face recognition.
- Provides limited multi-role institutional reporting.
- Requires external platforms for many academic processes.

1.1.1.3 MOSS MOSS is a source-code similarity detection service used to detect plagiarism in programming submissions [5].

Features

- Detects similarity across code submissions.
- Supports multiple programming languages.
- Helps instructors review plagiarism cases.
- Generates comparative similarity reports.
- Is widely used in programming education.

Limitations

- Does not schedule exams.
- Does not monitor students during live sessions.
- Does not manage labs or workstations.
- Has no attendance or face verification support.
- Requires a separate system for exam administration.

1.1.1.4 ExamSoft ExamSoft is a digital assessment platform supporting secure exams and analytics [6].

Features

- Supports secure digital exam delivery.
- Allows offline testing workflows.
- Provides assessment analytics.
- Supports question and exam management.
- Maintains institutional records.

Limitations

- Not specialized for lab workstation control.
- Limited seat and machine mapping.
- Weak focus on programming lab workflows.
- AI monitoring is not core feature.
- Requires customization for departments.

1.1.1.5 Moodle Moodle is a learning management system supporting course delivery and assessment [7].

Features

- Supports quizzes and assignments.
- Provides user management.
- Allows plugin extensions.
- Supports communication features.
- Maintains course records.

Limitations

- Not built for lab exams.
- Requires plugins for security.
- No machine-level tracking.
- Limited real-time invigilation.
- Weak incident evidence handling.

1.1.1.6 Proctorio Proctorio is an automated online proctoring system [8].

Features

- Browser lockdown support.
- Automated monitoring.
- Identity verification.
- Session recording.
- LMS integration.

Limitations

- Focused on remote exams.
- No lab resource allocation.
- Limited workstation control.
- Depends on external systems for results.
- Needs customization for institutions.

1.1.1.7 Honorlock Honorlock is an AI-based proctoring solution [9].

Features

- Identity verification.
- Browser monitoring.
- Video review support.
- AI-based detection.
- LMS integration.

Limitations

- Focus on remote exams.
- No lab scheduling.
- Limited workstation mapping.
- External result handling.
- Not focused on coding labs.

1.1.1.8 Blackboard Learn Blackboard Learn is an LMS platform for academic management [10].

Features

- Supports quizzes and grading.
- Role-based management.
- Communication tools.
- Centralized records.
- Integration support.

Limitations

- Not designed for lab exams.
- Needs add-ons for security.
- No machine monitoring.
- Limited incident logging.
- Requires customization.

1.1.1.9 Canvas LMS Canvas LMS supports course-level assessments [11].

Features

- Quiz and assignment support.
- Grading workflows.
- Role-based access.
- Third-party integrations.
- Communication tools.

Limitations

- Not standalone exam system.
- Needs lockdown tools.
- No workstation control.
- Limited invigilation.
- Weak evidence handling.

1.1.1.10 Google Classroom Google Classroom is a lightweight class management system [12].

Features

- Assignment management.
- Communication tools.
- Google integration.
- Easy to use.
- Cloud-based access.

Limitations

- No secure exam support.
- No browser lockdown.
- No invigilation dashboard.
- Limited logging.
- No lab control features.

The application review shows that existing systems usually emphasize one dimension of the problem, such as browser lockdown, remote proctoring, plagiarism checking, general LMS support, or formal assessment delivery. The gap remains the absence of one institution-focused platform that combines secure access, face verification, AI-supported monitoring, lab allocation, submission control, result handling, and reporting for practical computer-lab examinations.

Table 1.2: Application Comparison

Application	Lock	ID	AI Mon.	Sched.	Result	Lab Ctrl.	Ref.
Respondus LockDown Browser	✓	✗	✗	✗	✗	✗	[4]
Safe Exam Browser	✓	✗	✗	✗	✗	✗	[5]
MOSS	✗	✗	✗	✗	✗	✗	[5]
ExamSoft	✓	✓	✗	✓	✓	✗	[6]
Moodle	✗	✗	✗	✓	✓	✗	[7]
Proctorio	✓	✓	✓	✗	✗	✗	[8]
Honorlock	✓	✓	✓	✗	✗	✗	[9]
Blackboard Learn	✗	✗	✗	✓	✓	✗	[10]
Canvas LMS	✗	✗	✗	✓	✓	✗	[11]
Google Classroom	✗	✗	✗	✗	✗	✗	[12]
Proposed System	✓	✓	✓	✓	✓	✓	This Work

1.1.2 Future System Usage Analysis

The future system is expected to be used as an operational platform across the complete life cycle of practical laboratory examinations. Before the exam, administrators and teachers will use it to configure user roles, define examination sessions, assign laboratories, map students to approved machines, publish instructions, and confirm readiness. During the exam, students will use it to access the scheduled assessment under controlled conditions, while teachers and invigilators will rely on the monitoring dashboard to observe status changes, review alerts, and intervene when necessary. After the exam,

teachers and administrators will use it to review submissions, examine incidents, verify attendance, manage results, and generate structured reports.

In a practical institutional deployment, the system will be especially valuable in courses involving programming, software construction, data structures, operating systems, database laboratories, networking exercises, and related disciplines where each student works at a computer and produces digital outputs. It can help standardize operational conduct across sections and semesters and reduce dependence on improvised processes.

Another important future-use dimension concerns accountability and evidence. The proposed system will support a disciplined evidence trail through structured logs and linked session records, strengthening institutional memory and supporting academic disputes, result review, and quality evaluation.

1.2 Objectives

The overall objective of the AI-Driven Lab Exam Monitoring and Management System is to design and implement an integrated platform that improves the planning, supervision, control, and post-exam accountability of practical laboratory examinations. This overall objective is operationalized through a set of specific objectives that guide both the software solution and the academic direction of the project.

The first objective is to digitize and centralize the administration of practical laboratory examinations so that scheduling, user management, laboratory allocation, and session configuration can be handled consistently from one platform. The second objective is to provide invigilators and instructors with a stronger real-time awareness of the ongoing exam through status monitoring, anomaly alerts, and organized session visibility. The third objective is to create a dependable submission and evidence-handling workflow so that academic decisions can later be supported through structured records instead of improvised documentation.

The fourth objective is to improve fairness and procedural reliability by enforcing controlled access, time-bound execution, and consistent handling of student activity in the lab environment. The fifth objective is to reduce the burden of manual coordination placed on instructors and administrators by automating recurring operational tasks and by presenting consolidated reporting after exam completion. The sixth objective is to develop the project in a way that is academically meaningful, technically feasible within FYP constraints, and extensible for future institutional use.

1.3 Problem Statement / Limitations

Practical laboratory examinations are often conducted in environments where the technical challenge of the assessment is matched by an equally serious management challenge. In many institutions, there is no single platform that coordinates scheduling, invigilation, access control, workstation-level tracking, incident recording, and post-exam reporting in a unified way. As a result, important parts of the process remain manual, fragmented, or weakly documented. This makes the examination workflow harder to supervise and more difficult to justify when concerns arise.

One of the most significant limitations of the current environment is the limited ability of invigilators to monitor many students simultaneously with confidence. In a crowded computer lab, human attention alone is often insufficient to maintain real-time awareness of who has started, who is inactive, who may be experiencing technical issues, and who may be deviating from expected behavior. Even when invigilators are experienced, the lack of a consolidated dashboard makes consistent situational awareness difficult.

A second limitation concerns evidence and traceability. In many practical exams, suspicious activity, submission problems, or timing disputes are difficult to reconstruct after the session has ended. Without structured logs and organized incident records, academic decisions may depend on fragmented observations rather than well-preserved evidence. A third limitation lies in the administrative workload caused by separate tools and manual coordination. Scheduling, attendance, monitoring, submission collection, and reporting may each require different artifacts or systems, increasing operational overhead and the possibility of inconsistency.

The proposed project itself also has development limitations. As an FYP, it will emphasize a well-structured prototype and academically defensible design rather than claiming full enterprise-scale deployment across all university departments. Advanced AI-driven anomaly detection may initially be represented through configurable monitoring logic and alert support rather than through a production-trained predictive model. Similarly, large-scale live deployment constraints such as network variability, institutional integration complexity, and long-term maintenance policies may remain outside the immediate prototype boundary. These limitations are acknowledged so that the project remains realistic while still delivering meaningful academic and operational value.

1.4 Benefits of Proposed System

The proposed system offers a set of integrated capabilities that directly address the operational difficulties of practical laboratory examinations. Role-based user management supports different workflows for administrators, teachers, invigilators, and students, which helps the institution align permissions with responsibilities and reduce unauthorized access to sensitive exam functions.

Exam creation and scheduling allow authorized academic staff to define practical exams, assign dates and durations, publish instructions, and map sessions to relevant users and labs. This improves planning consistency and reduces confusion before the session begins.

Lab and workstation control strengthens the connection between the digital workflow and the physical laboratory environment by associating sessions with approved machines and locations. This is especially important in practical exams, where the technical workspace matters as much as the question paper itself.

Secure exam execution gives students a controlled and time-bound environment that is aligned with the scheduled session. This reduces ambiguity in exam start and finish conditions and supports procedural fairness.

The live monitoring dashboard gives invigilators a centralized view of active students, session status, timing, and alerts. This directly addresses the supervision challenges that emerge when many students are simultaneously working in the lab.

Alerting and evidence logging preserve meaningful exam events and suspicious occurrences as structured evidence. This improves post-exam review and strengthens accountability.

Submission management organizes student work through a controlled workflow that links each file or output with the correct candidate and session. This reduces the risk of disorganized file collection and supports cleaner evaluation.

Instructor review and administrative reporting help departments review attendance, incidents, session histories, and submission records through consolidated reporting features. This improves institutional documentation and helps departments learn from each conducted session.

The proposed solution is therefore not simply a secure browser, not simply a submission portal, and not simply an academic reporting tool. It is an integrated environment deliberately tailored to the needs of practical laboratory examinations. By addressing planning, supervision, traceability, and reporting in one coherent platform, it aims to produce a system that is academically useful, operationally realistic, and institutionally defensible.

1.5 Intended Market of Product

The intended market of the proposed product consists of educational institutions that conduct supervised practical examinations in computer laboratories and require stronger operational control, monitoring, and post-exam accountability.

1.5.1 Target Audience

The primary target audience includes universities, colleges, and departments offering programs in computer science, software engineering, information technology, cybersecurity, data science, and related technical disciplines. These institutions regularly conduct lab-based examinations where students work on institutional machines and produce digital outputs within a fixed time window. They need a system that not only delivers an exam but also supports role management, student verification, monitoring, submission handling, result management, and reporting.

1.5.2 Institutional Environment

A second important market segment includes academic training centers, skill-development institutes, certification labs, and departmental examination cells that manage scheduled practical tests for batches of students. Such environments require machine allocation, attendance control, restricted usage, and evidence-based review of exam sessions. The proposed system is suitable for these environments because it is designed around structured lab operations rather than general classroom administration alone.

1.6 Intended Users of the Product

The proposed system is intended for academic stakeholders who participate in practical laboratory examinations before, during, and after the scheduled session. Each user group interacts with the platform differently according to its responsibilities, but all of them contribute to a controlled and accountable exam workflow.

1.6.1 Administrator

The administrator uses the system to manage user accounts, assign roles and permissions, register laboratories, map available machines, configure institutional settings, and review overall exam records. This role is responsible for maintaining operational readiness, ensuring that approved users can access the appropriate modules, and generating administrative reports for audit and policy review.

1.6.2 Teachers and Students

Teachers use the system to create exams, upload instructions, schedule lab sessions, monitor live exam activity, review submissions, manage results, and study generated reports after the exam. Students use the system to log in to their assigned exam session, complete identity verification, access instructions, perform the practical task in a controlled environment, and submit the required files within the scheduled time. These two groups represent the core academic interaction supported by the platform.

1.7 Software Process Model

The development of the AI-Driven Lab Exam Monitoring and Management System follows the Agile process model. Agile is particularly suitable for this project because the system contains several interdependent modules, requires continuous refinement of workflows, and must respond to user feedback related to usability, security, and academic practicality. In projects involving dashboards, access roles, examination rules, event logging, and post-exam reporting, it is difficult to finalize every detail correctly at the very beginning. Agile provides a controlled but iterative way to build the system in increments, validate assumptions early, and refine the product across repeated feedback cycles.

1.7.1 Process Model Introduction

Agile is an iterative and incremental development model in which software evolves through repeated short cycles called sprints. Each sprint focuses on a manageable subset of work and produces a meaningful improvement in the system. Rather than postponing user feedback until the final stage, Agile encourages continuous interaction between planning, implementation, review, and refinement. This makes it highly appropriate for academic systems where stakeholders often understand their needs more clearly after seeing early prototypes and working workflows.

In the context of the proposed system, Agile supports a practical balance between technical implementation and institutional feedback. An early sprint can validate whether the role-based dashboard structure is clear, while a later sprint can refine monitoring logic, face verification flow, or reporting outputs based on observed weaknesses. Such iterative evolution helps reduce the risk of investing too much effort in features that are technically complete but operationally unsuitable. It also helps the project remain realistic within semester constraints while still demonstrating meaningful progress [1] [2].

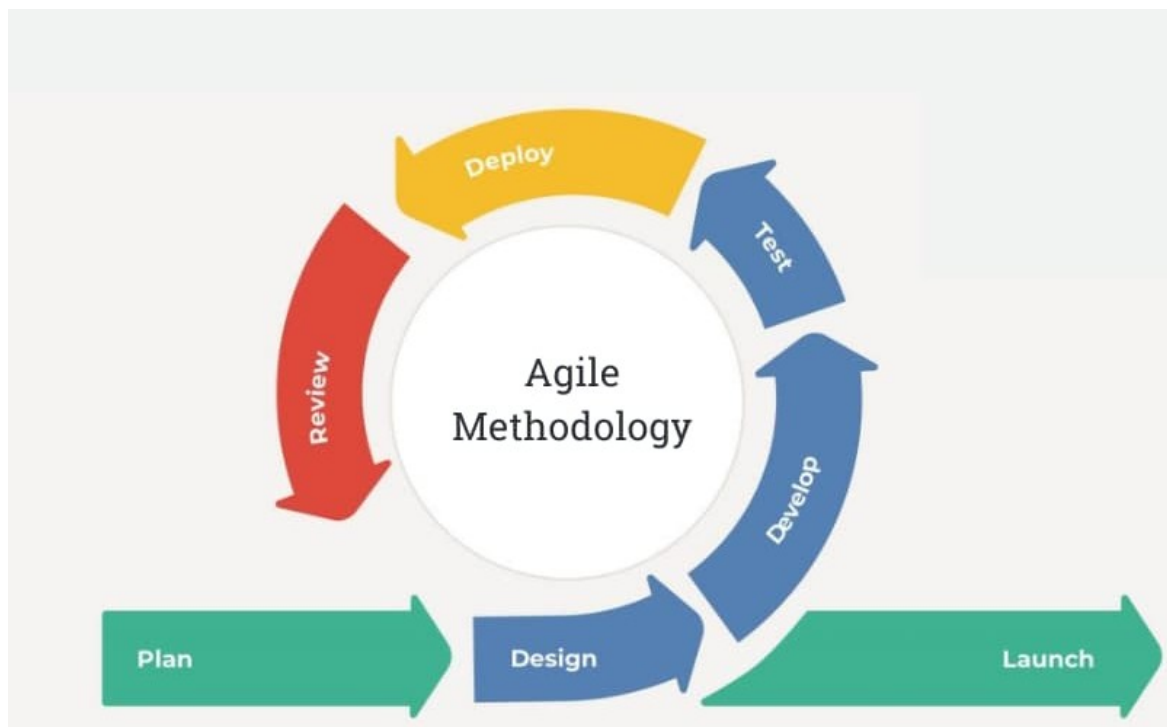


Figure 1.1: Detailed Agile Workflow with Sprint Feedback and Review Loops

1.7.2 Justification of Proposing the Process Model

Agile is chosen over a rigid sequential model because the proposed system is not a single-purpose utility; it is a multi-role academic platform whose usefulness depends heavily on workflow suitability. A strictly linear process would assume that scheduling logic, monitoring interface needs, alert behavior, evidence presentation, and reporting expectations can all be finalized with certainty before implementation begins. In reality, these design decisions benefit from iterative refinement. A dashboard that appears clear on paper may prove cluttered in a simulated invigilation scenario. A submission flow that seems simple in design may become confusing when aligned with real timing rules. Agile offers a way to respond to such issues before they become deeply embedded in the system.

Agile is also appropriate because the FYP itself develops under academic review conditions. Supervisor input, testing observations, and prototype demonstrations all contribute to the maturity of the final result. Instead of viewing change as a disruption, Agile treats controlled change as a normal and productive part of development. This aligns well with a student project in which requirements become sharper through research, modeling, and demonstration. Furthermore, the modular nature of the proposed system means that high-priority features such as login, scheduling, monitoring, and evidence logging can be developed first, while secondary enhancements can be refined later without destabilizing the project.

1.7.3 Steps of Process Model

The Agile process model for this project is organized into eight sprints. Each sprint spans four weeks, which is treated as an approximately thirty-day working cycle for planning, implementation, review, and refinement. The twelve project modules are distributed across these eight sprints so that each increment produces a meaningful and reviewable outcome.

Sprint 1 (Weeks 1–4): Requirement Analysis and System Planning. This sprint establishes the project backlog and defines the architecture baseline. The major modules covered are requirement analysis and project planning. Activities include stakeholder discussion, use-case identification, product backlog preparation, initial workflow sketches, and approval of the project scope.

Sprint 2 (Weeks 5–8): User Authentication and Role Management. This sprint focuses on secure access to the platform. The modules delivered are user authentication and role management. Activities include login design, account creation rules, session handling, access permissions, and dashboard routing for administrators, teachers, and students.

Sprint 3 (Weeks 9–12): Exam Scheduling and Question Management. This sprint addresses academic setup before the live exam. The modules delivered are exam scheduling and question or instruction management. Activities include exam creation, date and duration assignment, course mapping, instruction upload, and controlled publication of scheduled sessions.

Sprint 4 (Weeks 13–16): Laboratory Allocation and Workstation Control. This sprint develops the physical-lab administration layer. The modules delivered are laboratory allocation and workstation control. Activities include machine registration, lab mapping, seat assignment logic, approved-device association, and readiness checks before exam launch.

Sprint 5 (Weeks 17–20): Student Attendance and Face Recognition. This sprint strengthens candidate verification. The modules delivered are student attendance and

face recognition. Activities include attendance marking, pre-exam identity confirmation, candidate image matching flow, and validation of authorized student access to the assigned exam session.

Sprint 6 (Weeks 21–24): AI Monitoring and Incident Logging. This sprint covers active invigilation support during the exam. The modules delivered are AI-based monitoring and incident logging. Activities include live status tracking, suspicious-activity detection rules, alert creation, evidence capture, timestamped event records, and teacher-facing monitoring views.

Sprint 7 (Weeks 25–28): Result Management and Reporting. This sprint handles the post-exam management layer. The modules delivered are result management and reporting. Activities include submission review linkage, result entry, attendance summaries, incident summaries, report generation, and administrative retrieval of exam records.

Sprint 8 (Weeks 29–32): Integration, Testing, and Final Refinement. This sprint consolidates all previously completed work into one stable prototype. Activities include module integration, usability improvement, bug fixing, end-to-end testing, sprint review closure, documentation completion, and preparation of final academic deliverables for demonstration and report submission.

Chapter 2

SOFTWARE REQUIREMENTS SPECIFICATION

The Software Requirements Specification (SRS) defines the expected behaviour, quality attributes, user constraints, and operational boundaries of the proposed AI-Driven Lab Exam Monitoring and Management System. In academic software projects, the SRS performs a critical role because it transforms a general idea into a verifiable and structured description of what the product must accomplish. For this project, the SRS is especially important because the system is not limited to a simple examination portal. It must coordinate scheduling, restricted access, live supervision, evidence recording, controlled submission, instructor review, and administrative reporting in one coherent environment.

The specification also creates a shared understanding between the student development team, academic supervisor, and future stakeholders who may review the project. A well-written SRS reduces ambiguity by explaining not only which functions should exist, but also how those functions relate to institutional needs such as fairness, accountability, transparency, and operational continuity. In the context of lab examinations, unclear requirements can lead to serious problems. For example, if timing rules are not clearly defined, students may receive unequal working conditions. If monitoring requirements are not specified, suspicious incidents may be handled inconsistently. If reporting requirements are vague, academic staff may lack the evidence needed to interpret examination outcomes responsibly.

This chapter therefore explains the scope, audience, functional requirements, non-functional requirements, requirement elicitation techniques, and project time frame in a structured manner. The intention is to document the proposed system in sufficient depth so that later design and implementation decisions remain traceable to academic needs rather than being driven only by technical convenience. The chapter also supports future verification because every major feature described here can be translated into design elements, interface components, or test cases in later phases of the project.

2.1 Introduction

The introduction to the SRS provides the conceptual foundation for understanding the remainder of this chapter. It clarifies why the specification exists, who is expected to use it, and how the information should be interpreted during the later stages of development. For the proposed AI-Driven Lab Exam Monitoring and Management System, the introduction is necessary because the system combines educational administration, secure assessment control, and real-time monitoring workflows. These three dimensions make the project more complex than a conventional classroom application or static record-keeping tool.

The proposed system is intended to support practical laboratory examinations in a manner that is structured, transparent, and operationally manageable. Traditional lab-exam arrangements often depend heavily on manual invigilation, verbal coordination, file collection through ad hoc methods, and informal post-exam judgments about irregularities. Such practices can work on a small scale, but they become increasingly problematic when fairness, evidence preservation, and consistent policy enforcement are required. The

proposed system addresses this issue by specifying an integrated platform through which the exam can be planned, monitored, controlled, documented, and reviewed.

2.1.1 Document Scope

The scope of this SRS includes the description of all major user-facing and process-oriented requirements needed for the proposed system. It covers user authentication and role management, examination scheduling, student access control, live supervision support, irregularity alerting, event logging, controlled submission management, result-oriented reporting, and related quality expectations. It also includes the qualitative properties the system must exhibit, such as security, reliability, usability, maintainability, performance, and privacy sensitivity.

The document does not attempt to provide detailed algorithmic implementation at this stage. Instead, it explains what the system must do and the conditions under which it must do it. Such separation is important because requirements should remain stable enough to guide design, while implementation methods may evolve as prototypes are refined. The SRS therefore functions as a stable contract for expected product behaviour while allowing technical flexibility in how those behaviours are eventually realized.

2.1.2 Audience

This document is intended for several audiences. The primary audience includes the project team members who will use the SRS as the foundation for design, coding, testing, and report writing. A second important audience consists of the project supervisor and academic evaluators who need to determine whether the project scope is appropriate, coherent, and academically worthwhile. A third audience includes future stakeholders such as instructors, invigilators, lab administrators, or institutional reviewers who may examine the prototype or extend it later.

Each audience reads the SRS differently. Developers use it as a technical reference. Supervisors use it as a quality and scope-control document. Academic stakeholders use it to judge whether the system meaningfully addresses practical examination challenges. For this reason, the chapter is written in a way that combines technical clarity with institutional context. The specification should therefore be readable enough for academic review while remaining precise enough to guide development decisions.

2.2 Functional Requirements

Functional requirements define the explicit services, interactions, and behaviours that the system must provide in order to satisfy stakeholder expectations. In the proposed AI-Driven Lab Exam Monitoring and Management System, these requirements are interconnected because a practical lab exam must move in a controlled sequence from configuration to execution, monitoring, submission, and reporting.

Another important characteristic of these requirements is that they reflect academic control as well as software capability. The purpose of the system is not only to automate routine work, but also to provide exam governance in a form that is transparent, auditable, and fair. The following requirements are therefore written in operational sequence.

Table 2.1: Functional Requirements

Requirement ID	Description
FR1	The administrator shall create and manage user accounts for teachers, students, and other authorized users.
FR2	The system shall authenticate each user through a controlled login process.
FR3	The system shall assign role-based permissions according to administrator, teacher, and student responsibilities.
FR4	Teachers can create practical exams with course, date, duration, and instruction details.
FR5	The system shall assign each exam to a specific laboratory and approved workstation set.
FR6	Students can view only their scheduled exam session and related instructions.
FR7	The system shall mark student attendance before the exam begins.
FR8	The system shall verify student identity through face recognition before granting exam access.
FR9	Students can start the exam only within the allowed session time.
FR10	The system shall monitor live exam activity and present status updates to the teacher dashboard.
FR11	The system shall record alerts, suspicious events, and timestamped incident logs during the exam.
FR12	Students can submit required files or outputs through a controlled submission workflow.
FR13	Teachers can review submissions and manage results after exam completion.
FR14	The system shall generate attendance, incident, and result reports for administrative review.

The table above provides the compact list of the system's functional expectations. However, each requirement becomes more meaningful when interpreted within the actual practical-exam environment. The following narrative expands the major functional areas so that the intended product behaviour is clearer.

2.2.1 User and Access Management Requirements

The proposed system must support structured user administration because the academic examination process depends on role separation. Administrators manage users and configuration rules. Teachers create and review exams. Invigilators supervise ongoing sessions. Students participate in the examination and upload their work. If these responsibilities are not separated properly, the exam environment can become confusing or open to misuse. Therefore, account creation, role assignment, permission control, and secure authentication are fundamental requirements rather than optional convenience features.

The user-management component must also preserve operational simplicity. Academic environments often involve users with different technical skill levels. The system should therefore make it easy to activate or deactivate accounts, assign users to courses or exam sessions, and verify access privileges before an exam begins. These functions reduce the risk of confusion on exam day and strengthen institutional control.

2.2.2 Examination Planning and Scheduling Requirements

A major requirement of the system is the ability to plan practical examinations in an organized and policy-aligned manner. Authorized users should be able to define exam titles, descriptions, practical tasks, time limits, start and end windows, permitted locations, and related instructional details. This planning capability is essential because practical exams differ widely from one course to another. Some may require coding tasks, others software tool usage, and others file-based outputs. The system therefore needs scheduling flexibility without losing procedural discipline.

Scheduling requirements also include the ability to associate an exam with a specific laboratory, candidate group, or workstation context. In many institutions, different sections may share labs or conduct sessions at different times. Without structured scheduling support, such arrangements become difficult to manage fairly. The scheduling functions must therefore provide enough information for both teachers and invigilators to understand who is appearing, where they are appearing, and under what timing constraints.

2.2.3 Monitoring and Supervision Requirements

Monitoring is one of the defining features of the proposed system. Unlike ordinary learning platforms that focus mainly on content delivery or grading, this project aims to strengthen visibility during live practical exams. The system must therefore provide invigilators with a monitoring dashboard that displays active student sessions, timing information, progress state, and noteworthy alerts. This dashboard should support situational awareness rather than forcing invigilators to depend entirely on physical observation.

The monitoring requirement also includes the ability to register suspicious or irregular events. Depending on the final prototype implementation, such events may include inactivity anomalies, session interruptions, unauthorized state changes, or other configurable behaviours. The system should not make irreversible judgments automatically, but it should highlight events that deserve human review. In this way, the monitoring feature acts as a decision-support mechanism rather than a rigid disciplinary instrument.

2.2.4 Submission and Evidence Requirements

Practical exams often involve source files, outputs, project artifacts, screenshots, or documents that must be collected in a controlled manner. The system must therefore provide a clear submission workflow so that students know how to upload their required work before the session ends. A proper workflow minimizes ambiguity and helps prevent lost or misfiled submissions.

Evidence handling is equally important. The system should maintain structured linkage between the submission, the student identity, the assigned exam, the session timing, and the relevant event history. This evidence-oriented requirement supports academic fairness because it enables instructors or reviewers to interpret a submission within its full examination context. For example, if a technical interruption or suspicious event occurred, the evidence record should help explain that context during review.

2.2.5 Reporting and Post-Exam Review Requirements

The project must also support the post-exam phase, which is frequently neglected in simpler exam systems. After an examination has concluded, academic staff often

need more than raw student files. They need structured summaries showing attendance, submission completion, irregularity history, and session-level observations. Therefore, the system shall generate reports that help instructors and administrators review the exam in an organized manner.

Such reports are valuable for several reasons. They support internal quality review, help resolve student concerns, and improve future exam planning. They also elevate the system from being a real-time monitoring tool to being a broader management platform for practical examination governance.

2.3 Non-Functional Requirements

Non-functional requirements define the quality conditions under which the functional services must operate. In a project concerned with secure and fair academic assessment, non-functional properties are not secondary. If the system is difficult to use, invigilators may ignore it during the exam. If it performs poorly, the dashboard may become operationally irrelevant. If it is insecure, confidence in the entire assessment process may decline. For these reasons, the proposed system must satisfy multiple quality dimensions simultaneously.

2.3.1 Software Quality Attributes

The main software quality attributes for the proposed system are security, reliability, usability, performance, maintainability, and data integrity. Security is essential because the platform manages identities, exam schedules, student records, and potentially sensitive monitoring evidence. Reliability is equally important because a lab exam cannot pause simply because the system behaves inconsistently. Usability matters because administrators and teachers must interact with the platform under time pressure, while students need a clear and low-confusion exam interface.

Performance is also important because the system must respond quickly during login, attendance verification, exam launch, live monitoring, and file submission. Maintainability is relevant because the project may later be refined, extended, or integrated with broader institutional workflows. Data integrity remains critical throughout because schedules, attendance status, incident logs, results, and reports must remain consistent and auditable.

2.3.2 Other Non-Functional Requirements

Other relevant non-functional requirements include portability, privacy sensitivity, compatibility with university lab settings, and support for structured backup practices. The proposed system should operate within the technological realities of academic computer labs, where hardware and operating environments may vary. It should also preserve logs and exam records in a consistent manner so that important information is not lost due to technical disruption or later review needs.

Privacy considerations are particularly important when user activity, event logs, or submission-related evidence are retained. The system should therefore support responsible access control and discourage unnecessary exposure of sensitive student-related information. In addition, the platform should remain easy to document and explain within an academic environment, because FYP success depends not only on implementation but also on clarity of presentation and justification.

Table 2.2: Non-Functional Requirements

Category	Requirement
Security	The system shall enforce secure authentication, permission control, and protected access to examination records and evidence.
Reliability	The system shall preserve timing data, session status, event history, and submission records with minimal inconsistency risk.
Usability	The system shall provide clear interfaces for administrators, teachers, and students with minimum operational confusion.
Performance	The system shall respond quickly during login, exam launch, live monitoring, and submission actions.
Maintainability	The system shall support modular updates so that future improvements can be introduced without major redesign.
Privacy	The system shall restrict sensitive student, attendance, and monitoring information to authorized users only.
Compatibility	The system shall remain suitable for university computer-lab environments and commonly available institutional hardware.
Backup and Recovery	The system shall preserve important logs, reports, and result records through structured storage and recovery support.

The non-functional requirements show that the platform must be technically sound and institutionally dependable at the same time. Quality in this project cannot be judged only by whether a screen works. It must also be judged by whether the platform remains trustworthy when used under real examination pressure.

2.4 Requirement Gathering Techniques Used

The requirements for the proposed system were gathered through a multi-perspective process because lab examination problems are operational, academic, and technical at the same time. No single elicitation technique could capture the full picture. The project therefore used interviews, prototyping, and brainstorming sessions to define realistic requirements.

2.4.0.1 Interviews Interviews are a direct requirement elicitation technique in which stakeholders are asked structured questions to explain their current workflow, difficulties, and expectations. In this project, interviews were used to understand how teachers and academic staff currently schedule practical exams, supervise students, collect files, and review incidents.

Two key questions guided this technique. The first question asked what difficulties arise during practical lab exams when many students must be supervised at the same time. The second question asked what information teachers and administrators need after the exam for attendance verification, result handling, and academic review.

The outcomes of the interviews were clear and actionable. First, they highlighted

the need for role-based access and controlled scheduling. Second, they showed that live monitoring and incident logging are essential for exam oversight. Third, they confirmed that teachers need structured reporting after the session rather than scattered manual records.

2.4.0.2 Prototyping Prototyping is a requirement elicitation technique in which early interface sketches or mock workflows are shown to stakeholders so that hidden usability issues can be discovered before full implementation. In this project, simple interface ideas for login, scheduling, monitoring, and submission were considered to refine the system flow.

Two key questions guided this technique. The first question asked whether the proposed dashboards and screens were understandable enough for use during time-sensitive exam conditions. The second question asked whether the sequence from login to submission matched the actual working pattern of a lab exam.

The outcomes of prototyping improved the requirement quality in three ways. First, it clarified the need for a simpler teacher dashboard with visible student status. Second, it confirmed that students need a guided exam flow with clear timing and submission prompts. Third, it exposed the importance of integrating attendance, monitoring, and reporting instead of treating them as unrelated screens.

2.4.0.3 Brainstorming Session 1: Academic Workflow Discussion Brainstorming is a collaborative elicitation technique in which team members discuss ideas, problems, and possible solutions in multiple sessions to identify realistic and prioritized system requirements. In the first session, the discussion focused on the complete academic workflow, starting from exam creation and ending with final result review.

Two key questions guided this session. The first question asked which steps of the current practical exam workflow consume the most manual effort. The second question asked which modules are essential if the system is to provide value from the very first prototype.

The outcomes of the first brainstorming session established three priorities. First, exam scheduling and role management were identified as foundation modules. Second, lab allocation and attendance were recognized as necessary controls for a real computer-lab environment. Third, the team agreed that result management must remain connected to the earlier monitoring and submission steps.

2.4.0.4 Brainstorming Session 2: Monitoring and Reporting Discussion The second brainstorming session focused specifically on monitoring, identity verification, incident handling, and reporting. This session explored how AI-based alerts, face recognition, and evidence logging could be made useful within the practical limitations of an FYP project.

Two key questions guided this session. The first question asked which monitoring events should be recorded as meaningful evidence during a live exam. The second question asked what type of final reports would help teachers and administrators review the session efficiently.

The outcomes of the second brainstorming session refined the advanced modules of the project. First, the team prioritized face recognition before exam access rather than after the session. Second, it defined incident logs and alerts as central monitoring outputs. Third, it confirmed that attendance, incidents, and results should appear in one reporting layer so that post-exam review remains coherent.

2.5 Time Frame

The project time frame translates the Agile development plan into a structured schedule consisting of eight sprints. Each sprint lasts four weeks, allowing iterative development, review, and refinement while keeping the project aligned with academic deadlines.

Table 2.3: Project Time Frame (8 Sprints)

Sprint	Major Activities	Duration
Sprint 1	User and Access Management, Requirement Discovery, and Backlog Formation.	4 Weeks
Sprint 2	Exam Configuration, Scheduling, and Question & Instruction Management.	4 Weeks
Sprint 3	Laboratory Allocation and Workstation Control.	4 Weeks
Sprint 4	Student Exam Execution, Attendance, Face Recognition, and Submission Workflow.	4 Weeks
Sprint 5	Live Monitoring Dashboard, AI-based Alerts, and Real-time Supervision Features.	4 Weeks
Sprint 6	Evidence Logging, Incident Tracking, and Alert Management.	4 Weeks
Sprint 7	Result Management and Reporting Module Development.	4 Weeks
Sprint 8	System Integration, Testing, Validation, Documentation, and Final Refinement.	4 Weeks

This sprint-based schedule ensures continuous progress, regular supervisor feedback, and balanced development across all twelve modules of the proposed system. The final sprint focuses on integration and polishing so that the prototype is complete and academically presentable.

Chapter 3

SOFTWARE PROJECT PLAN

The Software Project Plan explains how the AI-Driven Lab Exam Monitoring and Management System will be organized, managed, tracked, and delivered throughout the final year project lifecycle. In software projects that involve multiple user roles, security-sensitive assessment workflows, real-time monitoring, evidence capture, and reporting obligations, planning is not a formality. It is the mechanism that holds the project together. Without a clear project plan, separate modules may be built in isolation, key deadlines may be missed, quality checks may become inconsistent, and documentation may fail to reflect the actual development journey.

For the proposed project, the plan is particularly important because the system integrates administrative control, invigilation support, and student-facing exam interaction within a single platform. Such integration creates dependency chains across modules. Scheduling logic affects monitoring. Monitoring affects evidence handling. Evidence handling affects reports. Reports affect post-exam academic decisions. Therefore, the project must be guided through a structured plan that balances ambition, feasibility, academic deadlines, and iterative improvement.

This chapter describes the deliverables, management approach, scheduling logic, managerial process assumptions, and risk management strategy that will guide the project. The overall aim is to ensure that development remains practical, measurable, and academically defensible from the first sprint to the final presentation.

3.1 Deliverables of the Project

Project deliverables are the concrete outputs that demonstrate whether the project is progressing in a meaningful and measurable way. In the context of a final year project, deliverables are not limited to the final software prototype. They also include planning artifacts, design descriptions, chapter documentation, diagrams, testing evidence, and presentation material. These deliverables help the team measure progress, align expectations with the supervisor, and ensure that the project remains grounded in documented academic work rather than ad hoc development.

For the proposed AI-Driven Lab Exam Monitoring and Management System, deliverables are especially important because the project combines practical software construction with report-based academic justification. A deliverable must therefore contribute either to system realization, project control, or academic evaluation. The following subsections explain the major deliverables in a more detailed manner.

3.1.1 Requirements and Analysis Deliverables

The first class of deliverables includes the problem analysis, literature grounding, requirement specification, workflow interpretation, and chapter documentation produced during the early stage of the project. These deliverables are important because they establish why the project is needed and what the system is expected to accomplish. They also act as a baseline against which later design and implementation decisions can be judged. If the analysis deliverables are weak, later development may become technically active but academically unfocused.

3.1.2 Design Deliverables

Design deliverables include the system model, process model illustration, work breakdown structure, scheduling diagrams, comparison tables, and any architecture-oriented descriptions used to convert requirements into a buildable plan. These outputs clarify how the system should be structured and how the development process should unfold. They reduce ambiguity by transforming general ideas into visual and textual planning artifacts. In this project, design deliverables are particularly important because the system must coordinate several interdependent modules such as scheduling, monitoring, submission management, and reporting.

3.1.3 Prototype and Module Deliverables

The major software deliverable of the project is a functional prototype of the AI-Driven Lab Exam Monitoring and Management System. However, the prototype itself is composed of smaller module-level deliverables. These include role-based login and dashboard access, exam creation and scheduling, candidate-session management, invigilator monitoring support, event logging, controlled submission handling, and report-oriented review support. Each module contributes to the completeness of the overall system, and each one must be sufficiently mature for the prototype to appear coherent rather than fragmented.

3.1.4 Testing and Validation Deliverables

Testing-related deliverables include structured validation of module behaviour, review of user flows, checking of reporting outputs, and confirmation that scheduling and monitoring logic work in a consistent manner. In an academic project, testing deliverables are essential because they show that the prototype has been examined critically rather than merely assembled. They also make it easier to explain which parts of the project were validated and which parts remain open to future improvement.

3.1.5 Documentation and Presentation Deliverables

The project report, final presentation, supervisor review responses, and supporting explanations are also critical deliverables. These outputs help communicate the purpose, structure, limitations, and contribution of the project. In many FYPs, a strong system can still be undervalued if the documentation does not explain its significance clearly. Therefore, written and presentation deliverables must be treated as central project outputs rather than end-stage formalities.

3.1.6 Gantt Chart

The Gantt chart visually represents the timing and overlap of major activities such as requirement analysis, sprint planning, interface design, module development, testing, and final review. In this revised plan, the chart is aligned with the eight sprint structure defined in the Agile process model so that the implementation schedule and the written process model remain consistent.

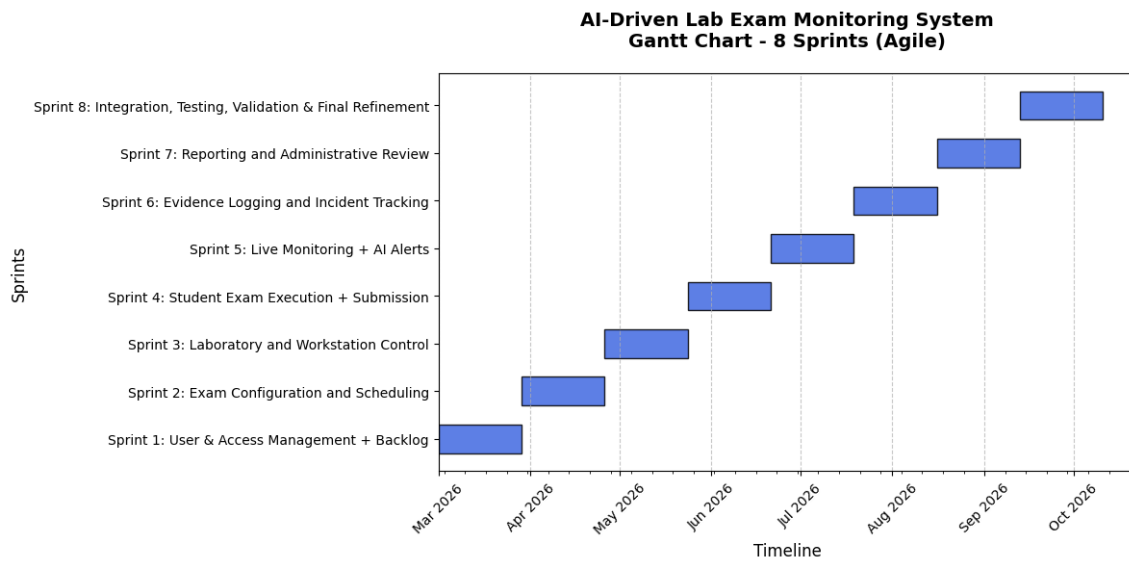


Figure 3.1: Project Gantt Chart Based on Eight Agile Sprints

The chart should be read together with the sprint descriptions because each four-week sprint represents a focused development increment with specific modules and review outcomes. When viewed in that way, the Gantt chart becomes a practical coordination tool rather than only a visual schedule.

3.1.7 Work Breakdown Structure

The work breakdown structure decomposes the project into manageable work packages. It helps the team see how the final system emerges from smaller responsibilities such as requirement analysis, dashboard planning, scheduling logic, monitoring logic, submission handling, reporting, testing, and documentation.

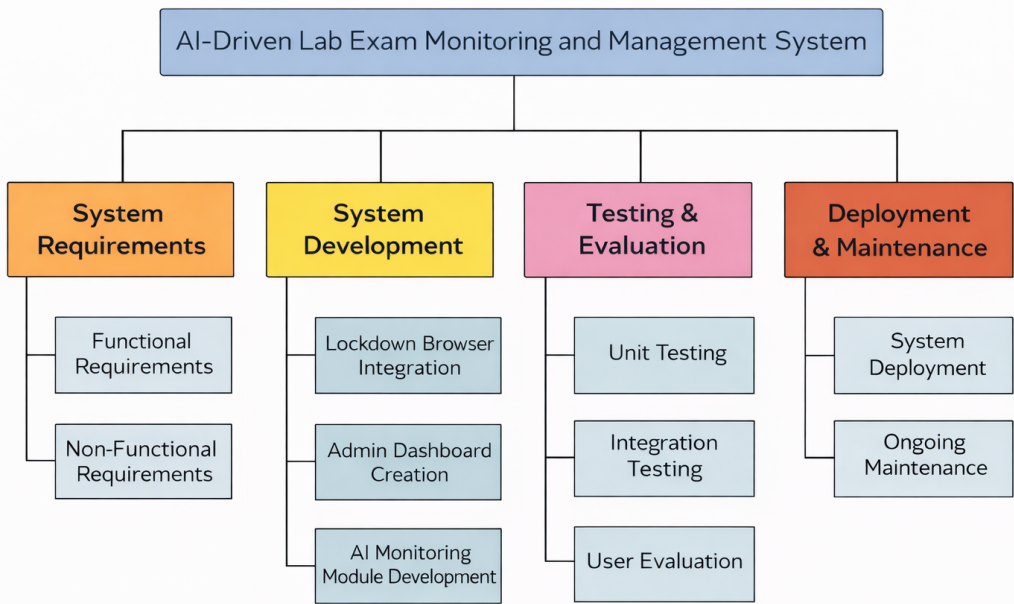


Figure 3.2: Work Breakdown Structure for AI-Driven Lab Exam Monitoring System

Figure 3.2: Work Breakdown Structure of the Proposed Project

The WBS is especially helpful in team coordination because it reduces ambiguity around what must be completed and how tasks relate to one another.

3.1.8 Critical Path Method

The critical path method identifies the chain of activities that most directly determines whether the project can finish on time. In this FYP, activities such as requirement stability, design completion, core-module development, integration, testing, and final documentation are part of that path. The critical path perspective is valuable because it helps the team recognize that some delays are more dangerous than others.

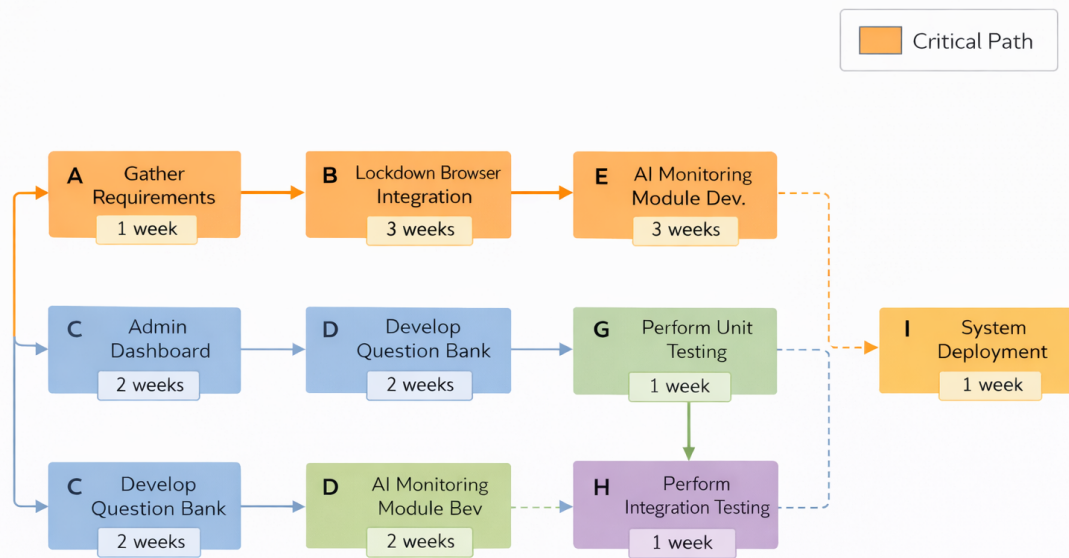


Figure 3.3: Critical Path of the Proposed Project

Figure 3.3: Critical Path of the Proposed Project Activities

3.1.8.1 Schedule Control and Review Schedule control in this project depends on regular sprint review, comparison of completed tasks against planned outcomes, and early adjustment when a task begins to slip. This approach is more useful than passive calendar tracking because it links time management with actual deliverables such as working modules, diagrams, documentation updates, and testing evidence.

3.2 Project Risk Management

Risk management is the process through which potential problems are identified, assessed, and prepared for before they seriously affect project success. In the proposed AI-Driven Lab Exam Monitoring and Management System, risk management is especially important because the project involves technical integration, real-time workflow assumptions, multiple stakeholder expectations, and strict academic deadlines. Risks do not need to become actual failures if they are recognized early and managed in a disciplined way.

3.2.1 Risk Identification

Several categories of risk may affect the project. Technical risks include integration difficulties between modules, instability in monitoring features, or data-handling complexity. Schedule risks include delays in design completion, incomplete sprint outcomes, or documentation falling behind implementation. Academic risks include misunderstanding stakeholder needs, insufficient justification in the report, or weak alignment between written chapters and the evolving prototype. Resource risks may include limited access to realistic testing conditions, hardware constraints, or tool-related limitations.

3.2.2 Risk Analysis

Risk analysis helps the team determine which risks deserve the most attention. A risk is not important only because it is possible; it becomes important when its likelihood combines with meaningful impact. For example, a small wording correction in a chapter

may be likely but low impact, whereas instability in a monitoring feature during late-stage integration may be both moderately likely and highly disruptive. The team therefore evaluates risks in terms of both probability and seriousness.

Table 3.1: Risk Likelihood and Impact Assessment

Risk	Likelihood	Impact	Explanation
Requirement instability	Medium	High	Frequent refinement of academic expectations may require redesign of chapters, figures, or system workflows.
Module integration difficulty	Medium	High	Scheduling, monitoring, and reporting modules may not combine smoothly without rework.
Documentation delay	High	Medium	If report writing lags behind development, final academic quality may suffer even if the prototype improves.
Testing constraints	Medium	Medium	Limited access to realistic lab scenarios may reduce confidence in workflow validation.
Time pressure near submission	High	High	Delays in one critical module may compress testing and revision time across the whole project.

3.2.3 Risk Mitigation Plan

Risk mitigation involves planning responses before problems grow. Requirement instability can be reduced through early scope clarification and regular supervisor review. Integration risk can be reduced through modular development and early testing of interfaces between key components. Documentation delay can be reduced by writing chapters incrementally throughout the project rather than postponing them. Testing constraints can be reduced through scenario-based validation even when full real-world deployment is not possible. Time pressure can be reduced through milestone tracking and careful attention to critical-path activities.

3.2.4 Risk Monitoring and Control

Risk management is not a one-time exercise performed only at the proposal stage. Risks must be monitored throughout the semester as the project evolves. New risks may appear when modules become more detailed or when feedback changes priorities. Therefore, the team should review risk status regularly, especially after each sprint or milestone. Continuous risk monitoring supports more stable project execution and reduces the likelihood that late-stage surprises will undermine the final deliverable.

References

- [1] NCBI, “Computer-based testing in higher education,” Available: <https://pmc.ncbi.nlm.nih.gov/articles/PMC12874087/>
- [2] MIT Open Learning, “Computer-based testing: secure, efficient, cost-effective,” Available: <https://openlearning.mit.edu/mit-faculty/residential-digital-innovations/computer-based-testing-secure-efficient-cost-effective>
- [3] ProductPlan, “Product Scope,” Available: <https://www.productplan.com/glossary/product-scope/>
- [4] Respondus, “LockDown Browser,” Available: <https://web.respondus.com/he/lockdownbrowser/>
- [5] Safe Exam Browser, “About Safe Exam Browser,” Available: https://safeexambrowser.org/about_overture_n.html
- [5] Stanford University, “MOSS: A System for Detecting Software Similarities,” Available: <https://theory.stanford.edu/aiken/moss/>
- [6] ExamSoft, “Learning Assessment Tools & Software,” Available: <https://examsoft.com/>
- [7] Moodle, “Moodle and Assessment,” Available: <https://moodle.com/moodle-and-assessment/>
- [8] Proctorio, “The Learning Integrity Platform,” Available: <https://proctorio.com/>
- [9] Honorlock, “What is Online Proctoring? Types, Features, and Integrations,” Available: <https://honorlock.com/blog/what-is-proctored-testing/>
- [10] Anthology, “Blackboard Learn,” Available: <https://www.anthology.com/products/teaching-and-learning/learning-effectiveness/blackboard/blackboard-learn>
- [11] Instructure, “Canvas LMS,” Available: <https://www.instructure.com/canvas>
- [12] Google for Education, “Google Classroom,” Available: <https://edu.google.com/workspace-for-education/classroom/>
- [13] Atlassian, “Project deliverables,” Available: <https://www.atlassian.com/work-management/project-management/project-deliverables>